제2회 모의고사 · 기출 유사

문제지 · A4 세로 2단 · 소문항별 답안란

이름 _____ 날짜 _____

60분 · 17문항 · 100점. 한국어 또는 의미가 같은 의사코드로 답할 수 있다. 인덱스·동률·구간 조건은 각 문항을 따른다. ↳는 인쇄용 줄바꿈이다.

1. DP 상태의 정보 [5점]

0-1 배낭에서 best(i,c)는 i번 이후 물건으로 남은 용량 c에서 얻는 최대 가치를 뜻한다. 그런데 캐시의 키를 i 하나만 사용하였다.

① [2점] 서로 다른 어떤 두 상태가 같은 키로 섞이는가? 예를 들어 쓰시오.

② [3점] 이 캐시가 잘못된 이유와 올바른 캐시 키를 쓰시오.

답안 · 1번

①

답안 · 1번

②

2. 절반 결과 재사용 [5점]

다음은 x^n을 계산하는 코드이다. n≥1이고 곱셈·나눗셈·나머지 연산은 상수 시간으로 센다.

① [2점] 시간 재귀식,

② [2점] 타이트한 시간 복잡도,

③ [1점] 재귀 스택의 추가 공간을 쓰시오.

```
def power(x,n):
    if n == 0: return 1
    half = power(x,n//2)
    value = half*half
    return value*x if n%2 else value
```

답안 · 2번

①

②

답안 · 2번

③

3. 해독 가능한 코드 [5점]

문자 A,B,C에 각각 0,01,11을 배정했다.

① [2점] 이 코드표는 prefix-free인가? 이유를 한 문장으로 쓰시오.

② [3점] 이진 코드 트리에서 실제 문자를 모두 잎에만 두면 prefix-free가 되는 이유를 쓰시오. “유일 해독 가능”과 “prefix-free”를 동일한 정의로 쓰지 마시오.

답안 · 3번

①

②

4. 스위치의 상태 복구 [5점]

시계 값 0,1,2,3은 각각 12시,3시,6시,9시이다. 시계는 [0,1,2,3]이고 스위치 하나는 0번과 2번 시계만 움직인다.

```
clocks = [0,1,2,3]
def push():
    for i in (0,2):
        clocks[i] = (clocks[i]+1)%4
for _ in range(2): push()
print(clocks)
for _ in range(2): push()
print(clocks)
```

① [2점] 두 출력 배열을 순서대로 쓰시오.

② [3점] CLOCKSYNC DFS에서 한 스위치의 네 분기를 모두 돌아 총 네 번 push하면 별도 undo 없이 상태가 복구되는 이유를 쓰시오.

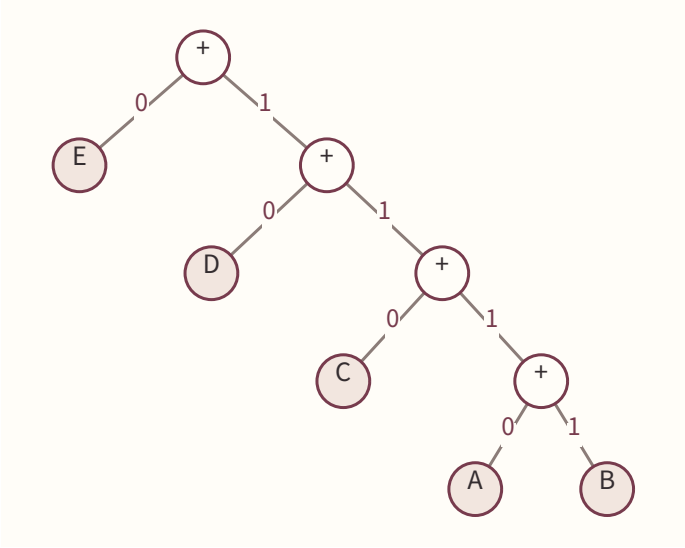
답안 · 4번

①

②

5. Huffman 디코더 [5점]

그림에서 왼쪽 간선은 0, 오른쪽은 1이다. 각 Node는 char,left,right 필드를 가지며 내부 노드의 char는 None이다. root는 그림의 루트이며 bits는 유효하고 완결된 비트열이다.



① [2점] 잎 판정 조건,

② [2점] 문자를 출력한 뒤 돌아갈 노드를 채우시오.

③ [1점] 이 코드표로 DE를 인코딩하시오.

```
def decode(root,bits):
    node, result = root, []
    for bit in bits:
        node = node.left if bit == '0' else node.right
        if ①____:
            result.append(node.char)
            node = ②____
    return ''.join(result)
```

답안 · 5번

① _____

② _____

답안 · 5번

③ _____

6. LIS 후보 비교 [5점]

A는 비어 있지 않은 수열이다. dp[i]는 A[i]에서 시작하는 엄격 증가 부분수열의 최대 길이이다. 동률 후보에서는 choices를 바꾸지 않는다.

① 유효한 다음 원소의 조건 [2점],

6. LIS 후보 비교 [5점] (계속)

② 후보 길이 [3점]를 채우시오.

```
def lis(A):
    n = len(A)
    dp, choices = [1]*n, [-1]*n
    for i in range(n-1,-1,-1):
        for j in range(i+1,n):
            if ①____:
                candidate = ②____
                if dp[i] < candidate:
                    dp[i], choices[i] = candidate, j
    return dp, choices
```

답안 · 6번

① _____

② _____

7. BOGGLE의 재방문과 캐시 [5점]

아래 BOGGLE은 상하좌우·대각선 8방향으로 이동하며 같은 칸을 다시 방문할 수 있다. word는 비어 있지 않다. k는 현재 맞출 문자의 인덱스이다.

① 캐시 키 [3점],

② 다음 문자 인덱스 [2점]를 채우시오.

```
def has_word(board,word,y,x):
    H, W, cache = len(board), len(board[0]), {}
    directions = [(dy,dx) for dy in (-1,0,1) for dx in (-1,0,1)
                  if (dy,dx)!=(0,0)]
    def search(y,x,k):
        if not (0 <= y < H and 0 <= x < W): return False
        if board[y][x] != word[k]: return False
        if k == len(word)-1: return True
        key = ①____
        if key not in cache:
            cache[key] = any(search(y+dy,x+dx,②____)
                             for dy,dx in directions)
        return cache[key]
    return search(y,x,0)
```

답안 · 7번

① _____

② _____

8. 완전탐색 호출 횟수 [5점]

스위치가 6개인 CLOCKSYNC를 각 스위치당 4개 선택으로 완전탐색한다. 성공 여부와 무관하게 모든 가지를 탐색하며 메모이제이션은 없다. 루트 깊이는 0이다.

① [2점] 리프 수

② [2점] 기저 호출까지 포함한 전체 노드 수

③ [1점] 최대 깊이를 쓰시오. 지수·Σ 형태를 허용한다.

8. 완전탐색 호출 횟수 [5점] (계속)

제공 공식: 1+b+⋯+b^d=(b^(d+1)−1)/(b−1).

답안 · 8번

① _____

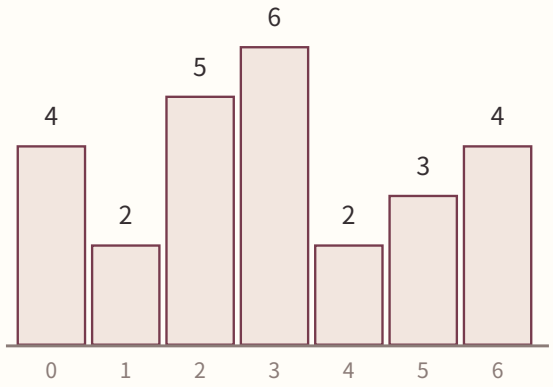
② _____

답안 · 8번

③ _____

9. 울타리 넓이 읽기 [5점]

판자 높이는 [4,2,5,6,2,3,4], 너비는 모두 1이다.



아래 숫자는 0부터 시작하는 인덱스

① [3점] 최대 직사각형 넓이를 쓰시오.

② [2점] 그 넓이를 만드는 한 구간의 양 끝 인덱스(0부터)와 높이를 쓰시오.

답안 · 9번

① _____

② _____

10. POLY의 상태와 표 [10점]

P(n,f)는 전체 n칸 중 첫 행이 f칸인 강의의 세로 단조 폴리오미노 수이다. P(n,n)=1이고, 다음 행 길이가 s일 때 붙이는 위치 수는 f+s−1이다.

① [2점] P(4,2)를 구하시오. P(2,1)=P(2,2)=1을 사용해도 된다.

② [2점] P(4,1)=6, P(4,3)=7, P(4,4)=1일 때, 첫 행 길이 제한이 없는 전체 4칸 모양 수를 쓰시오.

③ [3점] 메모 구현의 곱셈 계수,

10. POLY의 상태와 표 [10점] (계속)

④ [3점] 표 구현에서 참조할 남은 칸 수를 채우시오. 이 문제의 작은 입력에서는 모듈러를 사용하지 않는다.

```
def poly(n,first,cache):
    if n == first: return 1
    if (n,first) not in cache:
        cache[n,first] = 0
        for second in range(1,n-first+1):
            cache[n,first] += (③____) * poly(n-first,second,
↳             cache)
    return cache[n,first]

def poly_table(N):
    dp = [[0]*(N+1) for _ in range(N+1)]
    for n in range(1,N+1):
        dp[n][n] = 1
        for first in range(1,n):
            for second in range(1,n-first+1):
                dp[n][first] +=
↳                 (first+second-1)*dp[④____][second]
    return sum(dp[N])
```

답안 · 10번

① _____

② _____

답안 · 10번

③ _____

④ _____

11. 구간을 한 칸씩 늘리기 [5점]

누적합 배열 없이 시작점마다 구간 합을 갱신한다. cost의 길이는 n, 1≤L≤n이다.

① 갱신된 합 [2점],

② 평균의 분모 [3점]를 채우시오. 여기의 끝점 j는 구간에 포함된다.

```
def festival(cost,L):
    n, answer = len(cost), float('inf')
    for i in range(n):
        running = 0
        for j in range(i,n):
            running = ①____
            if j-i+1 >= L:
                answer = min(answer, running / (②____))
    return answer
```

답안 · 11번

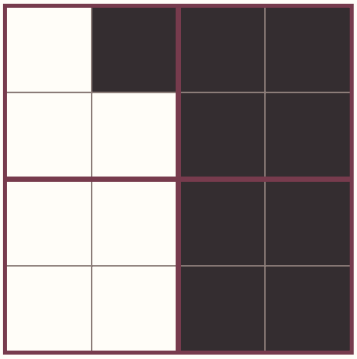
① _____

② _____

12. 그림과 재귀 문자열 [10점]

12. 그림과 재귀 문자열 [10점] (계속)

w=흰색, b=검정, x=네 영역 분할이다. 자식은 왼쪽 위·오른쪽 위·왼쪽 아래·오른쪽 아래 순서이다. 그림의 4×4 이미지는 xxwbwwbwb로 표현된다.



원본 4 × 4 이미지

① [2점] 이미지 전체를 상하 반전한 표현 문자열을 쓰시오.

② [2점] 최상위 네 부분의 순서만 바꾸고 내부를 그대로 두면 잘못되는 이유를 그림의 왼쪽 위 부분을 이용해 설명하시오.

③ [3점] 코드의 재귀 호출 인자,

④ [3점] 뒤집힌 네 자식의 결합식을 채우시오.

```
def flip(it):
    head = next(it)
    if head in 'wb': return head
    children = []
    for _ in range(4):
        children.append(flip(③____))
    return 'x' + ④____
```

답안 · 12번

①

답안 · 12번

②

12. 그림과 재귀 문자열 [10점] (계속)

답안 · 12번

③

④

13. 대진 순서의 선택 [5점]

같은 레이팅이면 우리 선수가 이긴다. bisect_left(a,x)는 정렬된 a에서 처음 x 이상인 위치를 반환하며 없으면 len(a)를 반환한다.

① 조건 [2점],

② 이기는 선수의 인덱스 [2점]를 채우고

③ [1점] match([6,2,8,5],[1,5,6,9])의 값을 쓰시오.

```
from bisect import bisect_left
def match(opponents,ours):
    ours, wins = sorted(ours), 0
    for rating in opponents:
        j = bisect_left(ours,rating)
        if ①____:
            ours.pop(0)
        else:
            wins += 1
            ours.pop(②____)
    return wins
```

답안 · 13번

①

②

답안 · 13번

③

14. 빠른 합의 호출 [5점]

S(n)=1+⋯+n이고 n≥1이다.

① [2점] 짝수 분기의 반환식을 half와 n으로 채우시오.

② [3점] fast_sum(11)의 호출 인자들을 최초 호출부터 기저 호출까지 순서대로 쓰고 총 호출 수를 쓰시오.

14. 빠른 합의 호출 [5점] (계속)

```
def fast_sum(n):
    if n == 1: return 1
    if n%2: return fast_sum(n-1)+n
    half = fast_sum(n//2)
    return ①____
```

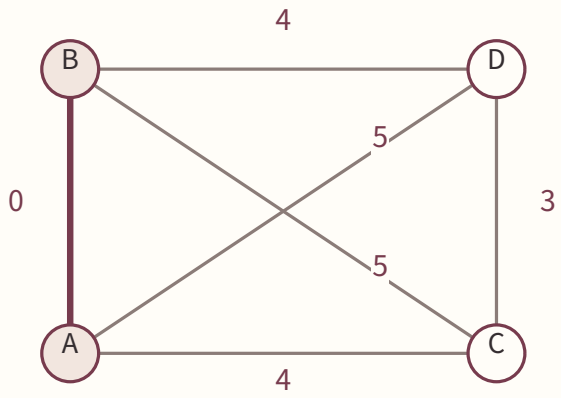
답안 · 14번

①

②

15. 기존 케이블과 Prim [5점]

건물 A=(0,0), B=(0,3), C=(4,0), D=(4,3)이다. A–B 케이블은 이미 설치되어 추가 비용이 0이다. 다른 두 건물 사이에는 직선 케이블을 설치할 수 있다. Prim에서 현재 선택된 정점은 {A,B}이고, 이번 문제의 minweight는 실제 길이이다.



A–B: 기존 케이블, 추가 비용 0

① [3점] C,D 각각의 (parent,minweight)를 쓰시오.

② [2점] 동률이면 이름이 앞선 정점을 고를 때, 다음 정점과 해당 추가 비용을 쓰시오.

답안 · 15번

①

답안 · 15번

②

16. k번째 LIS의 묶음 건너뛰기 [5점]

S=[1,3,2,5,4,6]이고 모든 값은 서로 다르다. L[i]는 i에서 시작하는 LIS 길이, C[i]는 그 길이를 만드는 수열 수이다.

i	0	1	2	3	4	5
S[i]	1	3	2	5	4	6
L[i]	4	3	3	2	2	1
C[i]	4	2	2	1	1	1

최대 길이의 LIS만 사전순으로 나열한다. 순번 k는 1부터 센다.

① [2점] k=3일 때 첫 값 1 뒤에서 값 2로 시작하는 분기를 건너뛰는가? 그 분기의 수열 수와 함께 답하시오.

② [3점] 세 번째 LIS의 원소 값을 쓰시오.

답안 · 16번

①

②

17. 조건이 다른 두 배낭 [10점]

용량은 15이며 물건은 각각 한 개씩 있다.

물건	A	B	C	D	E	F
무게	4	3	6	5	2	7
가치	19	12	21	20	7	22

① [3점] 분할을 허용하면 얻을 수 있는 최대 가치를 쓰시오. 분수 또는 소수로 답해도 된다.

② [3점] 분할을 허용하지 않는 0-1 배낭의 최대 가치를 쓰시오.

③ [2점] ②의 물건 목록을 쓰시오.

④ [2점] 가치/무게가 큰 순서로 통째로 넣는 전략이 0-1 최적해를 항상 보장하지 않는 이유를 한 문장으로 쓰시오.

답안 · 17번

①

②

답안 · 17번

③

④